Android - Unit 2

Building app UI

User Interfaces 2

Unit 2: Building App UI

1. Kotlin fundamentals
2. Add a button to an app
3. Interact with UI and state
    a. Met uitzondering van activity 5: **Write automated tests**

Unit 2 in de Google cursus bevat de te kennen stof, oefeningen en badges.
Deze slides en de slides op canvas geven meer uitleg over een aantal topics,
maar wat niet in de slides staat is net zo belangrijk!



PW1 Kotlin fundamentals (link)

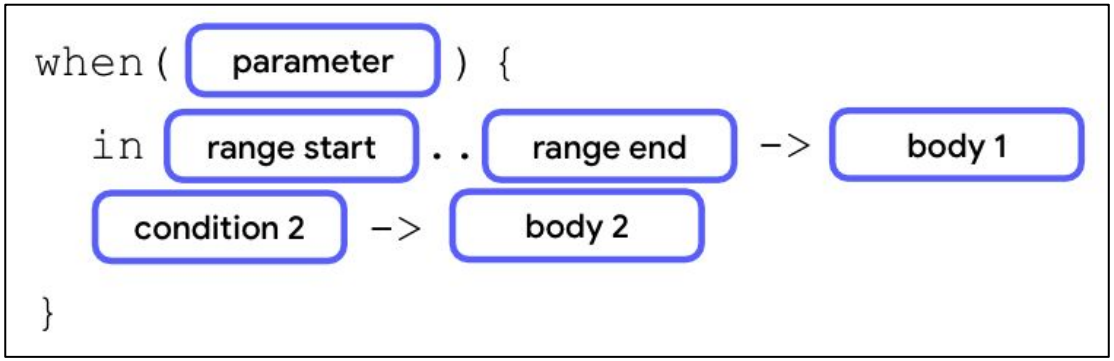


- Conditionals in Kotlin
- nullability in Kotlin
- Classes & objecten in Kotlin
- Function types & lambda expressions

Conditionals in Kotlin



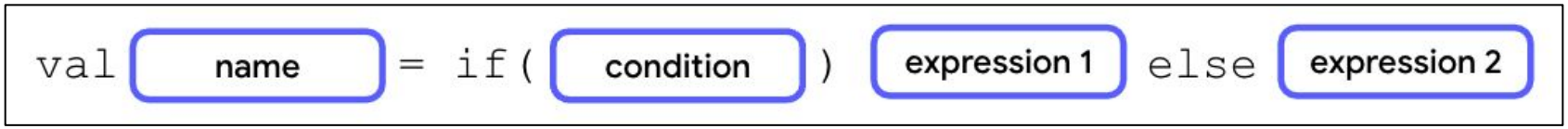
- Zie ook Slides Kotlin Lesson 1: Kotlin Basics : Conditionals
- `when` is vergelijkbaar met switch expression in Java 14+. Branches met
  - individuele waarden, door komma gescheiden lijsten
  - `in` ranges: ***`in from..toInclusive`***
  - datatype check met `is` en `else`.



Conditionals in Kotlin



- Conditionals (if, when...) zijn expressions: zij geven een waarde terug



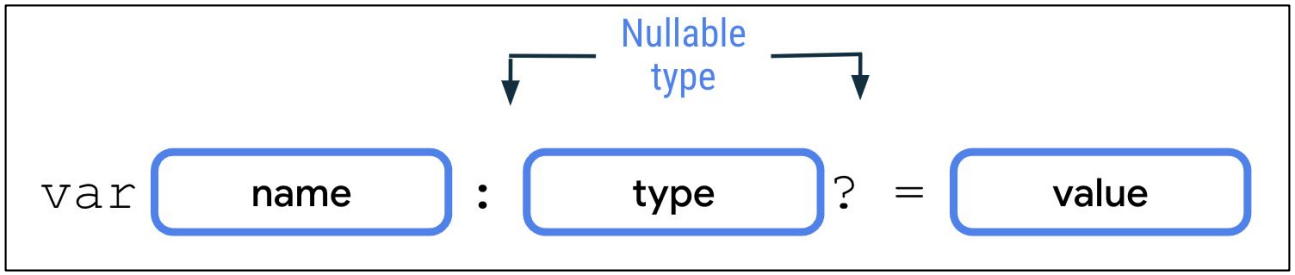
- Kotlin heeft **géén** ternary operator zoals java

```
var name = condition ? expression1 : expression2;
```

Nullability in Kotlin



- Zie ook Slides Kotlin Lesson 1: Kotlin Basics - Null Safety



- Een type is alleen nullable als je het expliciet `null` laat zijn (met **?**).
- Om methoden of eigenschappen van null variabelen te benaderen, moet gebruik je
  - **?.** (safe-call operators)
  - **!!** (not-null assertion operator, gooi exception indien null).
  - **?:** (elvis operator: gebruik alternatieve waarde indien null)

Klassen en objecten



- Zie ook Slides Kotlin Lesson 3: tot voor extension functions (tot slide 33)
- géén new keyword!
- `val`/`var` voor een klasse parameter voegt een property toe aan de klasse (= primaire constructor)
- `init` {blokken} kunnen gebruikt worden als code voor de primaire constructor
- secondaire `constructor` roept primaire constructor aan via `this`(...) (en ook `init` blokken)

```
class Square(val side: Double) {
  init {
    println(side * 2)
  }
}

val s = Square(10)
=> 20
```

Getters / setters



- Gebruik **`field`** om de backing property te benaderen in een get/set functie

```
var speakerVolume = 2
 set(value) {
   if (value in 0..100) {
     field = value
   }
 }
```

Inheritance



- enkel voor **`open`** classes, methods en properties
  - **`override`** keyword verplicht
- **`:`** operator om te erven van superklasse
- superklasse naam wordt gevolgd door de parameters van de superklasse constructor die door de primaire constructor aangeroepen wordt *(equivalent super(p) in Java)*

  ```
  class Derived(p: Int) : Base(p)
  ```
- een interface heeft geen constructor en de naam wordt dus nooit gevolgd door een (parameter lijst).
  - Een interface kan methode declaraties en implementaties (vergelijkbaar met java default methods) bevatten, maar geen state.
- abstract classes bestaan ook

Inheritance



- Een klasse kan meerdere interfaces implementeren.

```
// declares a method
interface A {
  fun callMe()
}

// contains a method
interface B {
  fun callMeToo() {
    println("From interface B")
  }
}

// gaat verder op volgende slide...
```

Inheritance



- Een klasse kan meerdere interfaces implementeren.

```
// implements two interfaces A and B
// implements abstract methods
class Child : A, B {
  override fun callMe() {
    println("Implements interface A")
  }
}

fun main(args: Array<String>) {
  val obj = Child()
  obj.callMe()
  obj.callMeToo()
}
```

Any / Visibility

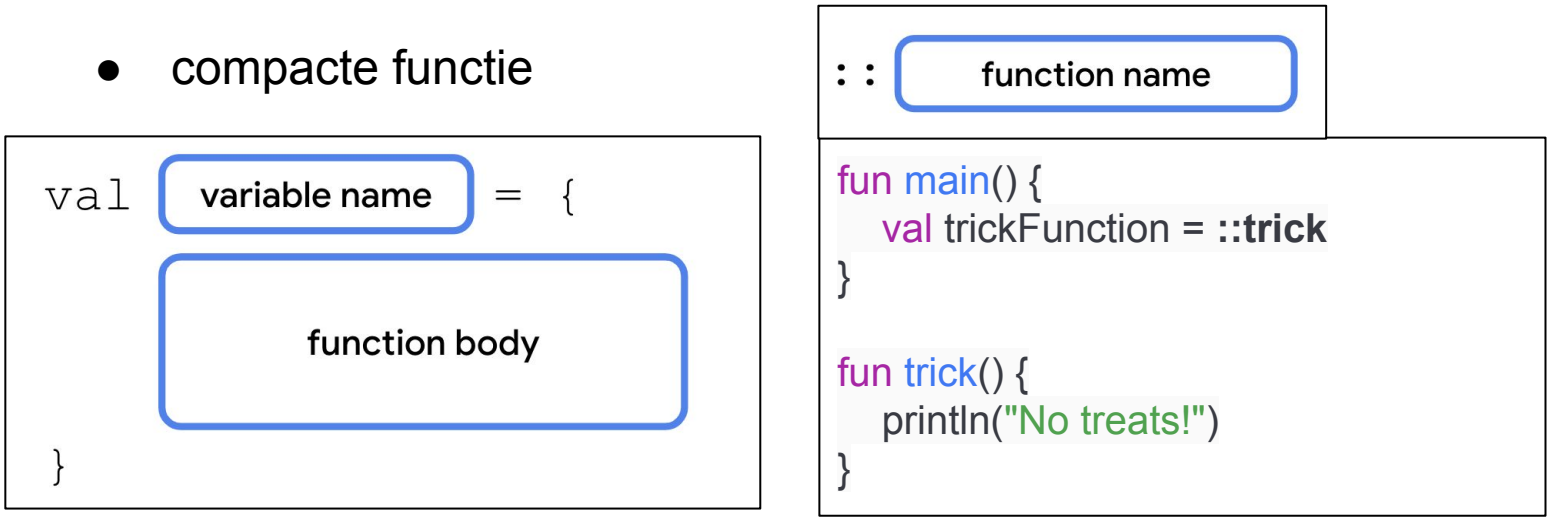


- **`Any?`** klasse is het equivalent van java's **`Object`** klasse
- Visibility modifiers
  - Alles is standaard `public`.
  - `private`
  - `internal` = bereikbaar in de module (compilatie eenheid).

Functies



- Zie ook [Slides Kotlin Lesson 2: Functions](): vanaf compact functions tot voor list filters



Functies



- Lambda's zijn zoals in java, maar
  - de volledige lambda, parameters en body, staat tussen **{**accolades**}**
    - rond de parameter**s** hoeven geen **(**haakjes**)**
  - als de lambda geen return statement bevat, wordt automatisch het resultaat van het laatste statement teruggegeven
- Functie als datatype:

(parameters (optional)) -> return type

```
val coins: (Int) -> String = {
    quantity ->
        "$quantity quarters"
}
```



https://kotlinlang.org/docs/lambdas.html

Functies



- Functie als een return Type

```
fun function name (): function type {
    // code
    return Name of another function
}
```

```
fun trickOrTreat(isTrick: Boolean): () -> Unit {
    if (isTrick) {
        return trick
    } else {
        return treat
    }
}
```

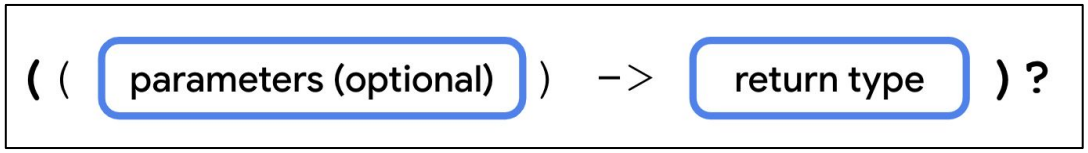
Functies



- Functie als parameter

```
fun trickOrTreat(isTrick: Boolean, extraTreat: (Int) -> String): () -> Unit {
    if (isTrick) {
        return trick
    } else {
        println(extraTreat(5))
        return treat
    }
}
```

- Nullable functie types
  - Een variabele of parameter kan een functie zijn of `null`



Functies



- Je kan een lambda rechtstreeks als parameter meegeven

```
val treatFunction = trickOrTreat(false,
    { coins -> "$coins quarters" }
)
```

- Als een lambda expressie slechts 1 parameter kan je it gebruiken
  - je kan dan de het "*parameter* ->" stuk van de lambda weglaten

```
val coins: (Int) -> String = { Parameters ->
    "$quantity quarters"
}
```

↓

```
val coins: (Int) -> String = {
    "$it quarters"
}
```

Functies



- Als de lambda de laatste parameter is, mag je hem **bij aanroep na** de `(parameterlijst)` zetten
- Documentatie van Box composable, laatste parameter is een functie die de inhoud van de composable:

```
@Composable
inline fun Box(
    modifier: Modifier = Modifier,
    contentAlignment: Alignment = Alignment.TopStart,
    propagateMinConstraints: Boolean = false,
    content: @Composable BoxScope.() -> Unit
): Unit
```

- Bij aanroep mag je de functie inhoud in het blok na de functie zetten:

```
Row(
    content = {
        Text("Some text")
        Text("Some more text")
        Text("Last text")
    }
)
```



```
Row {
    Text("Some text")
    Text("Some more text")
    Text("Last text")
}
```

Row *(parameterlijst) is nu leeg, je mag de* () *weglaten*

Delegates



- Ipv Getter en setter zal een Delegate te waarde managen
  - declaratie van een variabele gevolgd door het by sleutelwoord,

```
var name by delegate object
```

- Property delegates kunnen herhaalde codepatronen abstraheren. Eigenschappen kunnen worden geïmplementeerd met complexe logica zonder dat de implementatie van de eigenschap zelf complex wordt.
- Ingebouwde property delegates in Kotlin
  - `Lazy`
  - `Observable`
  - `remember`
- Je kan ook je eigen property delegates maken

https://medium.com/androiddevelopers/delegating-delegates-to-kotlin-ee0a0b21c52b

Oefening



Practice: Kotlin Fundamentals



Te behalen Badge Pathway 1:
https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-1

PW2 Add a button to an app (link)



- Interactieve Buttons & events

Code along: “Dice Roller app”

- codelabs hebben een renovate/all branch waar je updates naar nieuwere dependency versies vindt. Je kan natuurlijk ook zelf aanpassen in android studio

Practice: “Click Behaviour”



Te behalen Pathway 2:
https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-2

PW3 Interact with UI and state (link)



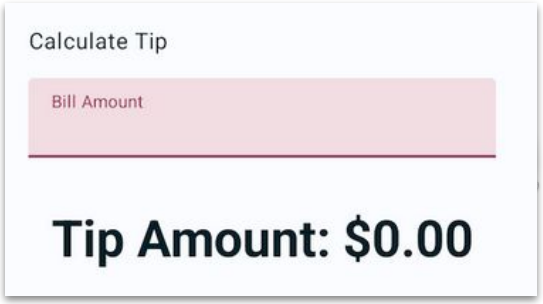
- Interactie met UI & state
  - Extra video en uitleg

`mutableStateOf` *Observed by Compose*
aanpassing van `var` *triggert recompositie*

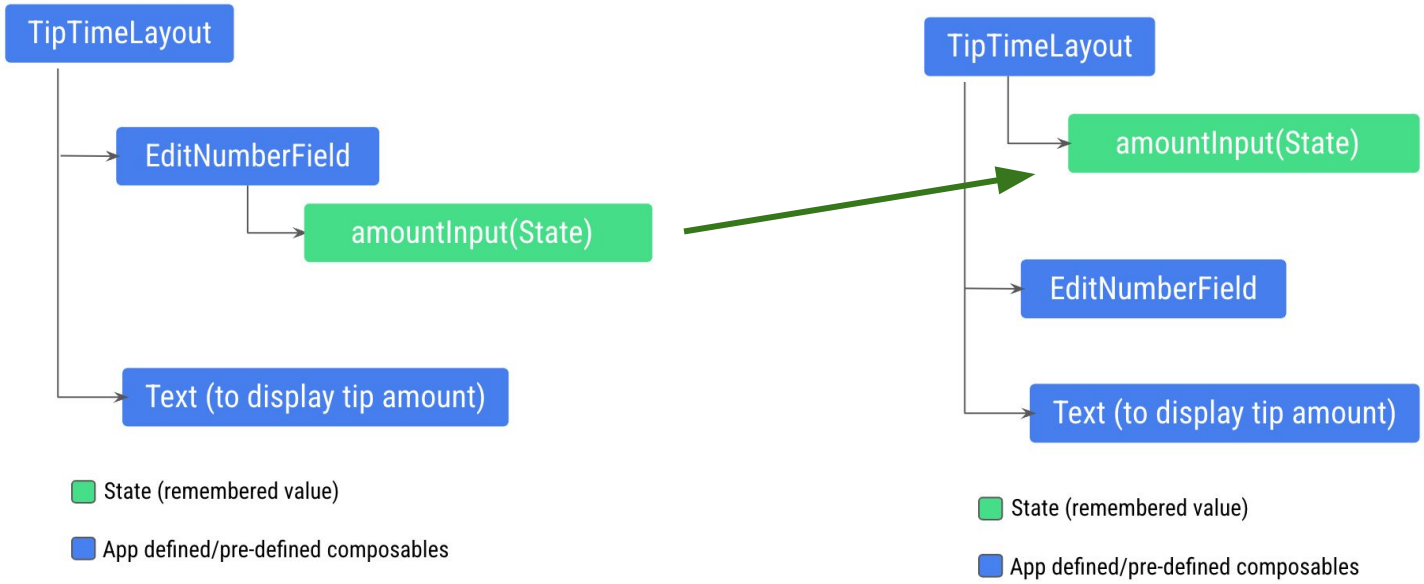
```
var amountInput by remember { mutableStateOf("")}
```

`remember` *delegate om waarde van var te behouden bij recompositie*

State hoisting



- What if you need to share remembered state between multiple components?
  - move the remembered state to a parent component
  - pass it to the children



Oefeningen



Verdieping *Geautomatiseerde tests*: bekijk even zodat je je badge kan halen, maar behoort verder niet tot de stof

- Code along: TipCalculator App
  - state and hoisting
- Code Along: Calculate a Custom Tip
  - keyboard action buttons
  - landscape scroll
  - Add leading icon to textfield
    - icon sets: materialdesignicons.com/, fonts.google.com/icons
- Project: “Art Space App”



Te behalen Pathway 3:
https://developer.android.com/courses/pathways/android-basics-compose-unit-2-pathway-3

Code conventies

- Minimum Android API version: **29,30 of 31**
- Hardcoded strings → strings.xml
- Clean code:
  - Lambda's
    - Verkorte syntax
    - Gebruik `it`
  - Property delegates (`by`)
- Composable bezitten best zo min mogelijk state! Sta toe dat de state wordt opgehaald wanneer dat zinvol is, bv in de API van de composable.
  - State hoisting: state verplaatsen naar hogere component